Optimisation des Performances en React

Objectifs

1. Comprendre les mécanismes de re-rendu de React et identifier les ralentissements.
2. Analyser les performances d'une application à l'aide de l'onglet Profiler des React Developer Tools.
3. Prévenir les rendus inutiles de composants enfants grâce à React.memo.
4. Mémoriser les références de fonctions avec le Hook useCallback.
5. Éviter les calculs lourds et répétitifs avec le Hook useMemo.
6. Comprendre quand il ne faut pas optimiser (éviter l'optimisation prématurée).

1. Comprendre le Mécanisme de Re-rendu

Avant de chercher à optimiser, il est essentiel de comprendre comment et pourquoi React décide de mettre à jour l'interface.

1.1 Quand un composant se re-rend-il ?

Un composant fonctionnel React s'exécute à nouveau (se "re-rend") dans trois situations principales :

1. **Changement d'état local :** La fonction setter (issue de useState) est appelée avec une nouvelle valeur.
2. **Changement de contexte :** Une valeur fournie par un Context API (ou Redux) auquel le composant est abonné est modifiée.
3. **Re-rendu du composant parent :** C'est la cause la plus fréquente de ralentissement. Par défaut, si un composant parent se re-rend, React re-rendra automatiquement tous ses enfants, petits-enfants, etc., même si leurs props n'ont pas changé.

1.2 Rendu Virtuel vs Peinture Réelle

Attention à ne pas confondre "Re-rendu React" et "Peinture du navigateur" :

- Le **re-rendu React** signifie simplement que la fonction du composant est rappelée pour générer un nouveau DOM Virtuel. C'est une opération en mémoire, généralement très rapide.
- La **peinture** est l'application des différences sur le DOM HTML réel par le navigateur. C'est l'opération la plus lente. React est déjà très optimisé pour éviter de peindre le DOM si le DOM virtuel n'a pas changé. Cependant, sur des arbres de composants très

profonds, le simple fait de recalculer le DOM virtuel de milliers d'enfants peut provoquer des saccades.

2. Diagnostiquer avec le Profiler

L'optimisation à l'aveugle est une mauvaise pratique. Il faut toujours mesurer avant de corriger. L'extension "React Developer Tools" propose un onglet **Profiler**.

2.1 Comment utiliser le Profiler

1. Ouvrez l'onglet Profiler dans les outils de développement du navigateur.
2. Cliquez sur le bouton "Record" (le rond bleu).
3. Effectuez l'action lente dans votre application (clic, frappe clavier).
4. Arrêtez l'enregistrement.
5. Analysez le graphique ("Flamegraph") : les barres les plus larges et jaunes signalent les composants qui ont mis le plus de temps à s'exécuter. Vous pouvez voir pourquoi un composant s'est rendu en le survolant (ex: "Le composant parent s'est rendu").

3. Prévenir les rendus en cascade avec React.memo

Pour casser la chaîne de rendu par défaut (où le parent force le rendu de l'enfant), on utilise **React.memo**.

3.1 Le principe de React.memo

React.memo est une fonction qui englobe un composant. Elle indique à React : "Si le parent se re-rend, regarde d'abord les props de cet enfant. Si les props sont strictement identiques au rendu précédent, ignore cet enfant et réutilise son dernier résultat."

3.2 Implémentation

JavaScript

```
import { memo } from 'react';

// Composant standard : se re-rend à chaque fois que son parent se re-rend
function StationDisplay({ name, status }) {
  console.log("Rendu de StationDisplay :", name);
  return (
    <div className="card">
      <h3>{name}</h3>
      <p>Statut: {status}</p>
    </div>
  );
}
```

```
// Composant mémorisé : ne se re-rend que si 'name' ou 'status' change
export const MemoizedStationDisplay = memo(StationDisplay);
```

3.3 Le piège de la comparaison par référence

React.memo compare les props de manière superficielle (comparaison d'égalité stricte). Si la prop est un nombre ou une chaîne de caractères, cela fonctionne parfaitement. Mais si la prop est un objet, un tableau ou une fonction, cela pose problème car deux objets identiques en apparence n'ont pas la même adresse en mémoire. C'est ici qu'interviennent useCallback et useMemo.

4. Conserver les références de fonctions avec useCallback

En JavaScript, chaque fois qu'une fonction composant s'exécute, toutes les fonctions déclarées à l'intérieur sont recréées avec une nouvelle adresse en mémoire.

4.1 Le problème

Si vous passez une fonction en prop à un enfant optimisé avec `React.memo`, cet enfant se re-rendra quand même à chaque fois, car il verra une fonction "nouvelle" à chaque rendu du parent.

4.2 La solution : useCallback

Le Hook **useCallback** permet de garder en mémoire la même référence de fonction entre plusieurs rendus, tant que ses dépendances ne changent pas.

JavaScript
```
import { useState, useCallback } from 'react';
import { MemoizedStationDisplay } from './StationDisplay';

export default function Dashboard() {
  const [count, setCount] = useState(0);

  // Sans useCallback : handleRefresh a une nouvelle référence à chaque clic sur
le compteur.
  // Avec useCallback : la référence reste identique tant que le composant est
monté (tableau de dépendances vide).
  const handleRefresh = useCallback(() => {
    console.log("Rafraîchissement des données de la station");
  }, []);
```

```
  return (
    <div>
      <button onClick={() => setCount(count + 1)}>Compteur global :
{count}</button>

      {/* Grâce à React.memo ET useCallback, cette carte ne clignotera plus au
clic du compteur global */}
      <MemoizedStationDisplay
        name="Toulouse"
        status="Online"
        onRefresh={handleRefresh}
      />
    </div>
  );
}
```

5. Mémoriser les calculs lourds avec useMemo

Parfois, le ralentissement ne vient pas d'un excès de re-rendus d'enfants, mais d'une opération mathématique ou d'un filtrage de tableau très complexe situé directement dans le corps du composant.

5.1 Le problème

Si votre composant liste 10 000 stations météo et doit les filtrer, ce calcul sera refait à chaque fois que l'utilisateur tape une lettre dans un champ de recherche, mais aussi s'il clique sur un bouton qui n'a rien à voir (comme ouvrir un menu).

5.2 La solution : useMemo

Le Hook **useMemo** exécute une fonction de calcul et mémorise son résultat. Lors du prochain rendu, si les dépendances n'ont pas changé, React saute le calcul et renvoie directement la valeur en mémoire.

JavaScript
```
import { useState, useMemo } from 'react';

export default function StationList({ stations }) {
  const [filter, setFilter] = useState('');
  const [theme, setTheme] = useState('light'); // Changer le thème ne doit pas
refaire le filtrage
```

```
  // Le filtrage massif n'est exécuté que si 'stations' ou 'filter' change.
  // Un changement de 'theme' n'aura aucun impact sur les performances ici.
  const filteredStations = useMemo(() => {
    console.log("Exécution du filtrage complexe...");
    return stations.filter(st =>
st.name.toLowerCase().includes(filter.toLowerCase()));
  }, [stations, filter]);

  return (
    <div className={`app-${theme}`}>
      <button onClick={() => setTheme(theme === 'light' ? 'dark' : 'light')}>
        Changer le thème
      </button>

      <input
        type="text"
        value={filter}
        onChange={(e) => setFilter(e.target.value)}
        placeholder="Filtrer..."
      />

      <ul>
        {filteredStations.map(st => <li key={st.id}>{st.name}</li>)}
      </ul>
    </div>
  );
}
```

6. L'Équilibre : Quand NE PAS optimiser

Une règle primordiale du développement logiciel est de fuir "l'optimisation prématurée".

1. **Les optimisations ont un coût :** `React.memo`, `useMemo` et `useCallback` obligent React à faire un travail supplémentaire en coulisses (allouer de la mémoire pour stocker les anciennes valeurs, et exécuter un algorithme de comparaison des dépendances à chaque rendu).
2. **Si le calcul est simple**, vérifier les dépendances avec `useMemo` peut prendre plus de temps que de simplement refaire le calcul.
3. **Approche recommandée :** Écrivez d'abord un code clair, simple et fonctionnel. Si l'application commence à souffrir de ralentissements perceptibles, utilisez le Profiler pour isoler le composant coupable, et appliquez ces Hooks uniquement à cet endroit précis.